chapter. We also experiment with the sizing of shards and see why allocating more replicas alleviates read performance problems.

We then move on to discuss how nodes communicate internally and form a cluster. We look at network settings and their importance in the second section of the chapter.

Any server with transactional and configuration data should be backed up regularly to avoid losing data in unforeseen circumstances. Elasticsearch provides functionalities to snapshot data at any time or as regularly as we wish and restore it as needed. We discuss the sophisticated snapshot and restoration functionalities in detail.

We also look at the advanced configuration for tweaking Elasticsearch properties. We examine the commonly used elasticsearch.yml configuration file and its contents, and we discuss how to change network settings, bump up the heap memory, and check the logs of a component at a trace level.

Finally, we discuss the role of the cluster master and how the cluster makes decisions based on a quorum, along with other details. We look at the split-brain scenario to understand the minimum number of master-eligible nodes required in a healthy cluster.

Productionizing Elasticsearch is an involved and expert task. Elasticsearch has many moving parts, and getting a handle on each is a huge task—but not impossible. While most of the features of Elasticsearch work out of the box, it is not enough to move the application to production; administrative tasks must also be taken care of. Many options must be tweaked and tuned to get Elasticsearch (or Elastic Stack) to a production state. Covering all these administrative tasks in one chapter would be tedious and impractical. However, I've included the common and important administrative functions that most developers and administrators must understand. Refer to the appropriate documentation for deep dives into functions not covered in this chapter. Let's start by learning about the scaling of clusters.

14.1 Scaling the cluster

Elasticsearch clusters can scale up to any number of nodes, from a single node to hundreds, based on the use case, data, and business requirements. Although we may work with a single-node cluster on a personal machine when learning about Elasticsearch, we seldom have a single-node cluster in production.

One reason we chose Elasticsearch is its resilience and fault-tolerance capabilities. We don't want to lose data when a node crashes. Fortunately, Elasticsearch can cope with hardware failures and recover from them as soon as the hardware is back online.

Choosing the cluster size is an important IT strategy for any organization, and multiple variables, factors, and inputs go into sizing Elasticsearch clusters for our data needs. Although we can add resources (memory or new nodes) to an existing cluster, it is important to forecast such demands.

In this section, we learn how to size and scale the cluster. We can add nodes to an existing cluster to increase read throughput or indexing performance. We can also downsize clusters by removing nodes, perhaps because the demand for indexing or read throughput has declined.